



v22 ▾



API

Update Guide

Select the options that match your update

Angular versions

From v.

21.0 ▾

To v.

22.0 ▾

Application complexity

Basic

Medium



Advanced

Shows all the information we have about this update.

Other dependencies



I use ngUpgrade to combine AngularJS & Angular



I use Angular Material



I use Windows

[Show me how to update!](#)

Guide to update your Angular application v21.0 → v22.0 for advanced applications

Before you update

You don't need to do anything before moving between these versions.

Update to the new version

Review these changes and perform the actions to update your application.

- ☐ In the application's project directory, run `ng update @angular/core@22 @angular/cli@22` to update your application to Angular v22.
- ☐ Run `ng update @angular/material@22`.
- ☐ Angular v22 requires Node.js v22.22.3 or v24.15.0 and later. Update your Node.js version to meet this minimum requirement. You can check your current version with `node --version`.
- ☐ Update your project to use TypeScript 6.0 or later. Versions older than 6.0 are no longer supported. Use `ng update` which will handle this automatically.
- ☐ Data-prefixed attributes (e.g., `data-*`) no longer bind to inputs or outputs. If you were relying on this behavior, use explicit property bindings instead (e.g., `[attr.data-value]="value"` or `[dataValue]="value"` for a component input).
- ☐ The compiler now throws an error when inputs, outputs, or model are binding to the same property/output. Review your component decorators and ensure no duplicate bindings exist.
- ☐ Safe navigation (`?.`) and nullish coalescing (`??`) now correctly narrow down nullable types in templates. This may trigger `nullishCoalescingNotNullable` and `optionalChainNotNullable` diagnostics on existing projects. Either fix the diagnostics by updating your templates, or temporarily disable them in your `tsconfig.json` under `angularCompilerOptions`.
- ☐ Angular expressions with optional chaining (`?.`) now return `undefined` instead of `null`. You can use the `$safeNavigationMigration()` magic function to revert to the previous behavior.
- ☐ `in` variables in template expressions now throw an error as it does in native JavaScript. If you have variables named `in` in your component or template, update your template expressions to use `this.in` or rename your variable.
- ☐ The `AnimationCallbackEvent.animationComplete` signature has changed. Update any code that depends on the old signature of this event. Review your animation event handlers and tests.
- ☐ If your application uses upload progress reporting through `HttpXhrBackend`, explicitly opt-in by using `provideHttpClient(withXhr())`. The default HTTP client no longer includes XHR support by default.

- ☐ The `reportProgress` option in HTTP requests is deprecated. Use `reportUploadProgress` or `reportDownloadProgress` instead for more explicit control over progress reporting.
- ☐ `provideRoutes()` has been removed. Use `provideRouter()` instead, or configure routes as a multi token using `ROUTES` if necessary. Update your application bootstrap configuration.
- ☐ If using AngularJS interoperability, replace deprecated `getAngularLib()` and `setAngularLib()` with `getAngularJSGlobal()` and `setAngularJSGlobal()` respectively.
- ☐ `ComponentFactoryResolver` and `ComponentFactory` are no longer available. Pass the component class directly to APIs like `ViewContainerRef.createComponent()` or use the standalone `createComponent()` function instead.
- ☐ `createNgModuleRef` has been removed. Use `createNgModule()` instead for dynamic module creation scenarios.
- ☐ Elements with multiple matching selectors now throw a compile-time error. Ensure your components use unique selectors and review any directives that might have conflicting selectors.
- ☐ Components with no `changeDetection` property defined are now `OnPush` by default. To maintain `Eager` (the previous default) change detection, explicitly set `changeDetection: ChangeDetectionStrategy.Eager` in your component decorator.
- ☐ `ChangeDetectorRef.checkNoChanges()` has been removed. In tests, use `fixture.detectChanges()` instead or verify your component state through other means.
- ☐ Leave animations are no longer limited to the element being removed. They now support nested animations scoped to component boundaries. Review your animation configurations if you relied on the previous scoping behavior.
- ☐ `paramsInheritanceStrategy` now defaults to `"always"` instead of `"emptyOnly"`. This means route parameters are inherited from all parent routes. To restore the previous behavior, explicitly set `paramsInheritanceStrategy: "emptyOnly"` in your router configuration.
- ☐ The `currentSnapshot` parameter in `CanMatchFn` and the `canMatch` method of the `CanMatch` interface is now required. Update any class implementations of `CanMatch` to include this required third argument.
- ☐ Hammer.js integration has been removed from Angular platform-browser. If you need touch gesture support, implement your own gesture detection or use an alternative library.

- ☐ The second argument of `appRef.bootstrap()` no longer accepts `any` type. Ensure the element you pass is not nullable and matches the expected type.
- ☐ Unused styles are now automatically removed when their associated `host` is dropped. Be aware that other DOM on the page may be affected if those styles are used by elements outside of Angular or if not using `ViewEncapsulation.Emulated`.
- ☐ The return type for `TitleStrategy.getResolvedTitleForRoute` has changed from `any` to a stricter type (e.g., `string | undefined`). Update your custom `TitleStrategy` implementations to match the new signature.
- ☐ Incremental hydration is now the default behavior for applications using Server-Side Rendering (SSR). Review your application if you relied on the previous non-incremental hydration behavior. You can use `withNoIncrementalHydration()` to restore the previous behavior if needed.
- ☐ The `fullTemplateTypeCheck` compiler option has been removed. Use `strictTemplates` instead to enable strict template type checking in your `tsconfig.json`.
- ☐ The `strictTemplates` compiler option now defaults to `true`. If your project was not using strict template type checking, you may see new compilation errors. Resolve these errors or explicitly set `strictTemplates: false` in your `tsconfig.json` to opt out.
- ☐ Webpack builders (`@angular-devkit/build-angular` and `@angular-devkit/build-webpack`) are now deprecated. Migrate to the `@angular/build` builders (esbuild/application) for your application builds.
- ☐ `CommonEngine` APIs from `@angular/ssr` are deprecated. Migrate to `AngularNodeAppEngine` or `AngularAppEngine` instead.
- ☐ `istanbul-lib-instrument` is now an optional peer dependency. If your project uses Karma with code coverage enabled, ensure `istanbul-lib-instrument` is explicitly installed.
- ☐ `ng serve` now gives the highest priority to the `PORT` environment variable. This value overrides any port configured in `angular.json` or provided via the `--port` flag.
- ☐ The `@angular-devkit/architect-cli` package is no longer available. Use the `architect` CLI tool from the `@angular-devkit/architect` package instead.
- ☐ The experimental `@angular-devkit/build-angular:jest` and `@angular-devkit/build-angular:web-test-runner` test builders have been removed.

After you update

You don't need to do anything after moving between these versions.

Social Media	Community	Resources	Community translations
Blog	Contribute	Press Kit	Azərbaycanca
X (formerly Twitter)	Code of Conduct	Roadmap	Español
Bluesky	Report Issues		Türkçe
YouTube	Google's DevLibrary		Русский
Discord	Angular Google Developer Experts		한국어
GitHub			日本語版
Stack Overflow			简体中文版
			正體中文版

Super-powered by Google ©2010-2026. Code licensed under an [MIT-style License](#) . Documentation licensed under [CC BY 4.0](#) . Built by Angular at v22.0.0+sha-8f06057.